# Spanner

## Final handover and implementation record

Riese Auto-Clinic: workshop booking, stock and invoicing
ITC327W Phase 3 (Assessment 3): System Development, Implementation and Final Demonstration

**Group Y "The Zealots"**
Submitted 30 October 2026

| Item | Where |
|---|---|
| GitHub | https://github.com/BlackZ3ro/Spanner-Riese-Auto-Clinic (branch `main`) |
| Live website | https://riese-auto-clinic.vercel.app |
| Mobile app | Flutter, source in `mobile/` |
| Backend | One shared Supabase project (migrations 0001 to 0011 in `supabase/migrations/`) |

Wisani Nkuna 222010787 · Gary van Wyk 222048911 · Keamogetse Kgosi 222003092 · Khumo Moremi 222062729 · Denny Mathye 221014861 · Ofentse Rammutla 223056856 · Breyton Rossouw 222069744

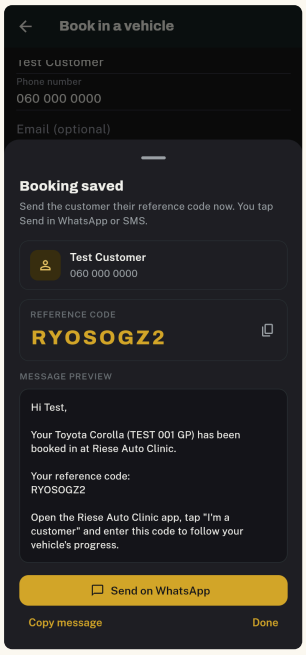
# 1. What was built

Spanner replaces Riese Auto-Clinic's paper and memory-based process. Mechanics and the front desk use a **Flutter mobile app** and an **ASP.NET Core 10 (Razor Pages) website**. Both talk to **one Supabase project** (PostgreSQL, Auth, Row Level Security, database functions, Realtime on the web). Customers need no account: they follow a repair with a reference code in the app or on the public `/Track` page.

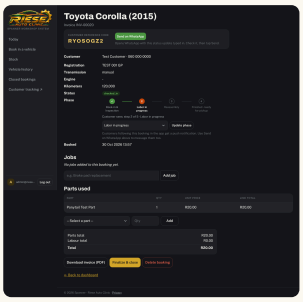
Neither app contains a service-role key. Both use the public anon key, and each staff request carries that person's own login, so database rules apply to every action. The website is hosted on Vercel; the mobile app runs on Android (and as a web build for the screenshots).

### Two shared processes, shown working in both directions

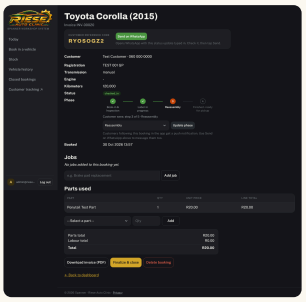
| # | Process | Flutter action | ASP.NET action | Shared data |
|---|---|---|---|---|
| P1 | Book-in, part and stock stay in step | Book in a vehicle, add a part and labor | The booking, code, customer and part are on the website; stock page shows the new quantity | `bookings`, `customers`, `booking_parts`, `stock_items` |
| P2 | Phase change reaches the customer | Customer enters the code and sees the phase; staff list updates | Staff move the phase, press Send on WhatsApp; `/Track` shows it | `bookings.phase`, `reference_code` |



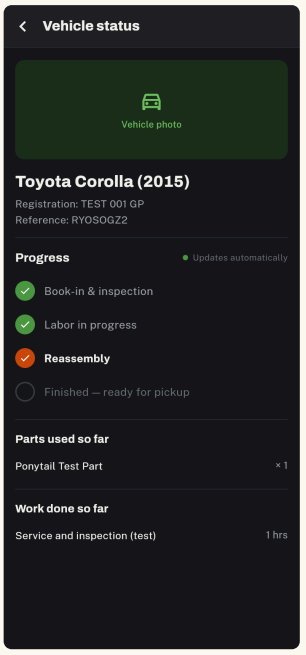

m09: booking created on the phone, reference code and WhatsApp message.




w04: the same booking, invoice number and part on the website.




w10: phase changed on the website.




m17: the phone shows the website's change.

# 2. Requirement-to-function status

Status as seen working on 30 October 2026 against the live system, with dummy data only. "Partly" means it works with a stated limitation.

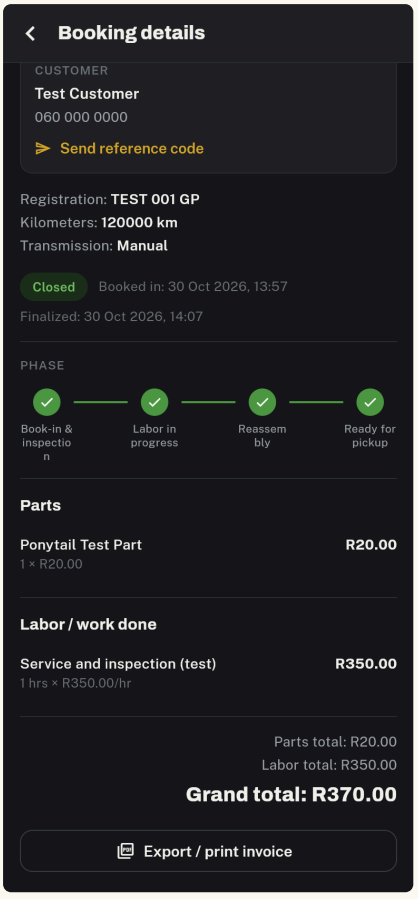
| ID | Requirement | Status | Evidence | Notes |
|---|---|---|---|---|
| FR-01 | Book in a vehicle (customer and vehicle details, validation) | **Built and shown** | Mobile m07-m10; web w05 | Both apps. Reference code and invoice number are generated by the database. |
| FR-02 | Assign jobs to a booking | **Built and shown** | Web w11 | Jobs on the website; labour is no longer kept on jobs (one ledger, see section 5). |
| FR-03 | Add and view stock items | **Built and shown** | Mobile m23; web w06 | Same stock table in both apps. |
| FR-04 | Link parts to a booking, stock drops | **Built and shown** | Mobile m11, m12, m24; web w04 | Web uses one database function (all-or-nothing); mobile makes three calls. |
| FR-05 | Record labour hours and rate | **Built and shown** | Mobile m15; website Labour section | One ledger since 30 Oct: both apps write the same table, `booking_labor`. |
| FR-06 | Job cost = parts + labour | **Built and shown** | Mobile m16; website booking page | Phone and website now read the same labour lines. Tested 30 Oct: R350 + R150 = R500 on the page and on the invoice. |
| FR-07 | Invoice PDF | **Built and shown** | Web w16 and invoice-INV-00020-test.pdf; mobile export | Both apps generate a PDF from the same labour lines. |
| FR-08 | Share / download the invoice | **Built and shown** | Web w16 | Download link on the web; export / print on the phone. |
| FR-09 | Changes seen by the other app | **Partly** | m18, w04, w10 | Web dashboard live-reloads. The phone needs a reload (L3). |
| FR-10 | Stock levels and low-stock highlight | **Built and shown** | m23, w03, w06 | |
| FR-11 | Search and list bookings, vehicle history | **Built and shown** | m06, m22, w13 | Web vehicle-history search by registration. No VIN column exists (approved change). |
| FR-12 | Staff login and roles | **Built and shown** | m04, m25, w01, w02, w15 | Supabase Auth; admin and mechanic. |
| FR-13 | Wikimedia vehicle reference image | **Partly** | m10 (placeholder) | Works on the web; did not load in the capture environment (L5). |
| FR-14 | Customer tracks a repair by reference code, no login | **Built and shown** | m02, m03, m17, w09, w14 | Via a database function only; no table access for customers. |
| FR-15 | Update booking phase | **Built and shown** | m13, m14, w10 | Same five phases in both apps. |
| FR-16 | Customer is notified of a phase change | **Partly** | m13; WhatsApp message file | WhatsApp click-to-chat works on both. Push needs a real Android phone and was not tested here (L4). |
| FR-17 | Finalize, reopen, delete restocks: admin only | **Built and shown** | m16, m20, m21, m26, w12, w15 | Hidden for mechanics in both apps; delete is database-enforced (0007). |
| FR-18 | Delete a booking and restock its parts | **Built and shown** | Run on the live system; stock returned from 2 to 3 | Admin only. |

**Non-functional requirements.** NFR-01 simple interface: validation messages and a four-step tracker (m07, w02). NFR-02 speed: checked in Phase 2 testing (pages in 0.4 to 2 s, invoice PDF 1.7 s). NFR-03 behaves when Supabase is unreachable: clean message, no crash (tested earlier). NFR-06 cross-browser: only Chromium was available; Firefox and Safari were **not** tested.

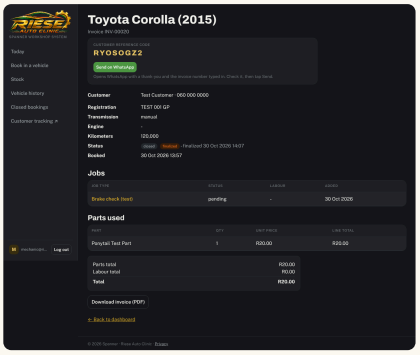
# 3. Backend, access rules and roles

| User | Allowed | Enforced by |
|---|---|---|
| Anyone not logged in | Nothing staff-only | Checked live 20 Oct: no table rows readable and the staff functions refuse (migration 0008) |
| Customer (no login) | Their own booking, by reference code | Security-definer function `get_booking_by_reference_code` only |
| Mechanic | Book in, parts, labor, phase, stock; no finalize, reopen or delete | Delete: database (0007). Finalize and reopen: database (0011). A mechanic's update to a closed booking changed 0 rows while the admin's identical update changed 1 (30 Oct) |
| Admin | Everything, including finalize, reopen, delete with restock | Admin check in the database |

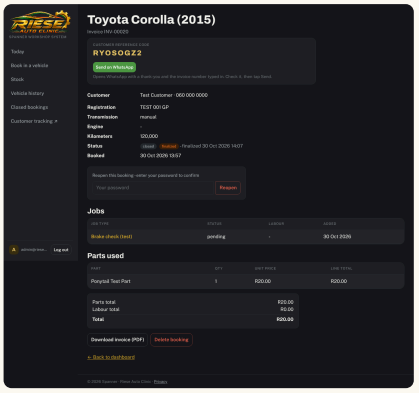
**Migration status.** All of 0001 to 0011 have been run on the shared project (the group confirmed 0009 and 0011 on 30 Oct; I confirmed 0008 and 0010 from outside the database).




m26: mechanic on a closed booking, no Reopen or Delete.




w15: same booking on the website as a mechanic.




w12: admin view, Reopen and Delete available.

# 4. Approved changes since Phase 2

| Change | Reason |
|---|---|
| Customer repair-status lookup by reference code (FR-14, Change 7) | Partly reverses Phase 1 Change 5. Michael Riese has confirmed it (30 Oct 2026). |
| One labour ledger: `booking_labor` (decided 30 Oct) | The website used to keep labour on `jobs`, so the two invoices could differ. The website now reads and writes `booking_labor`, like the phone; the customer tracking page and the delete function already used it. |
| Send on WhatsApp via free click-to-chat links, not the paid WhatsApp Business API | Mr G's feedback of 16 Oct; keeps the zero-budget rule. |
| Vehicle search by registration instead of VIN | No VIN column exists in the real schema. |
| Razor Pages instead of MVC | Approved design; each page has a view and a handler. |
| Website hosted on Vercel; look matched to the mobile app (dark theme) | Phase 3 presentation on a real address; one consistent look. |
| `created_by` on bookings (0010) | Ms Mbele asked who booked a car in. |

# 5. Known limitations

| # | Limitation | Effect |
|---|---|---|
| L1 | **Resolved 30 Oct.** Labour was in two tables (`booking_labor` on the phone, `jobs` on the website). | One ledger now. Older jobs that still hold hours and rate on the `jobs` row are not counted. The screenshots w04, w12 and w16 were taken before the change and show Labour R0. |
| L2 | Admin role comes from two places (mobile: `profiles.role`; web: an email list). | A role change must be made in both. |
| L3 | The mobile app does not update live and has no pull-to-refresh on staff lists. | Reload to see the other app's changes. |
| L4 | Push notifications and fingerprint login were not tested (need a real Android phone). | Unverified in this evidence. |
| L5 | Vehicle reference photo did not load in the capture environment. | Placeholder shown. |
| L6 | Mobile add-part is three calls, not one all-or-nothing function. | A failure midway could leave stock out of step. |
| L7 | Container hosting on Vercel is beta; the site sleeps after about 5 minutes idle. | Open it 2 minutes before the demo. Fallback: `dotnet run` on a laptop. |
| L8 | A Firebase service-account key was committed in September and again in October; it is still in git history. | It must be revoked in Google Cloud. |

# 6. How to run it (setup guidance)

### Website (`web/`)

Requires the .NET 10 SDK. Set the Supabase URL and the **anon key** (never the service-role key) as user secrets, then run it:

```
cd web
dotnet user-secrets set "Supabase:Url" "https://<project>.supabase.co"
dotnet user-secrets set "Supabase:AnonKey" "<anon key>"
dotnet run
```

On Vercel the same values are the settings `Supabase__Url`, `Supabase__AnonKey`, `DataProtection__Secret` (any long random text, shared by all containers) and `PublicSiteUrl`. See `project-docs/phase-3/hosting-on-vercel.md`.

### Mobile app (`mobile/`)

Requires Flutter 3.x. Create `mobile/.env` with `SUPABASE_URL` and `SUPABASE_ANON_KEY` only. Then `flutter pub get` and `flutter run` (Android), or `flutter run -d chrome` for the web build. Push needs Firebase configuration on Android only.

### Database

Run `supabase/migrations/0001` to `0011` in order in the Supabase SQL editor. Staff accounts are created in Supabase Auth with a matching row in `profiles`.

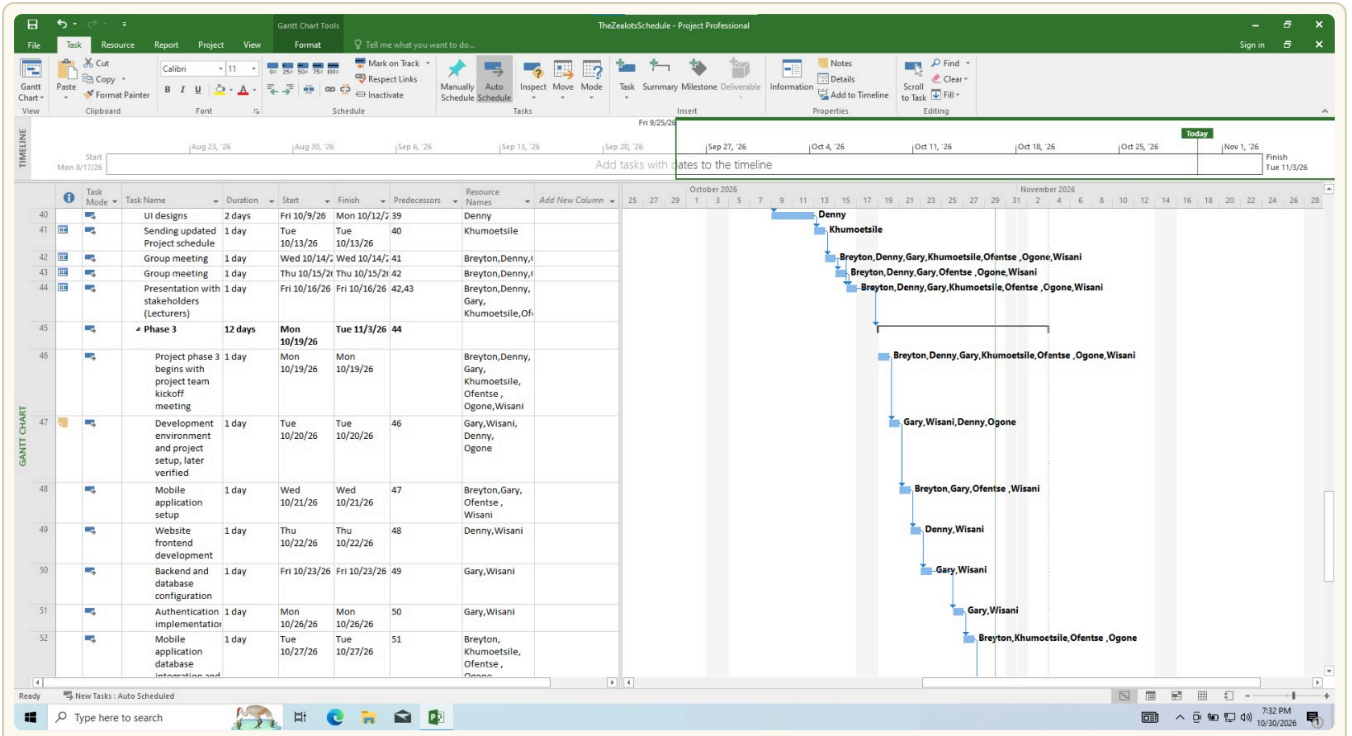
# 7. Team and contributions

| Member | Main contribution in Phase 3 |
|---|---|
| Gary van Wyk | Flutter app, Supabase set-up and migrations, WhatsApp send sheet, biometric login |
| Wisani Nkuna | ASP.NET site, Vercel hosting, evidence pack, tracker, coordination |
| Kea Kgosi | GitHub repository and access, recording shared processes |
| Khumo Moremi | Microsoft Project schedule |
| Denny Mathye | SRS content, access rules and limitations |
| Ofentse Rammutla | Web function checks and screenshots |
| Breyton Rossouw | Flutter function checks and screenshots |

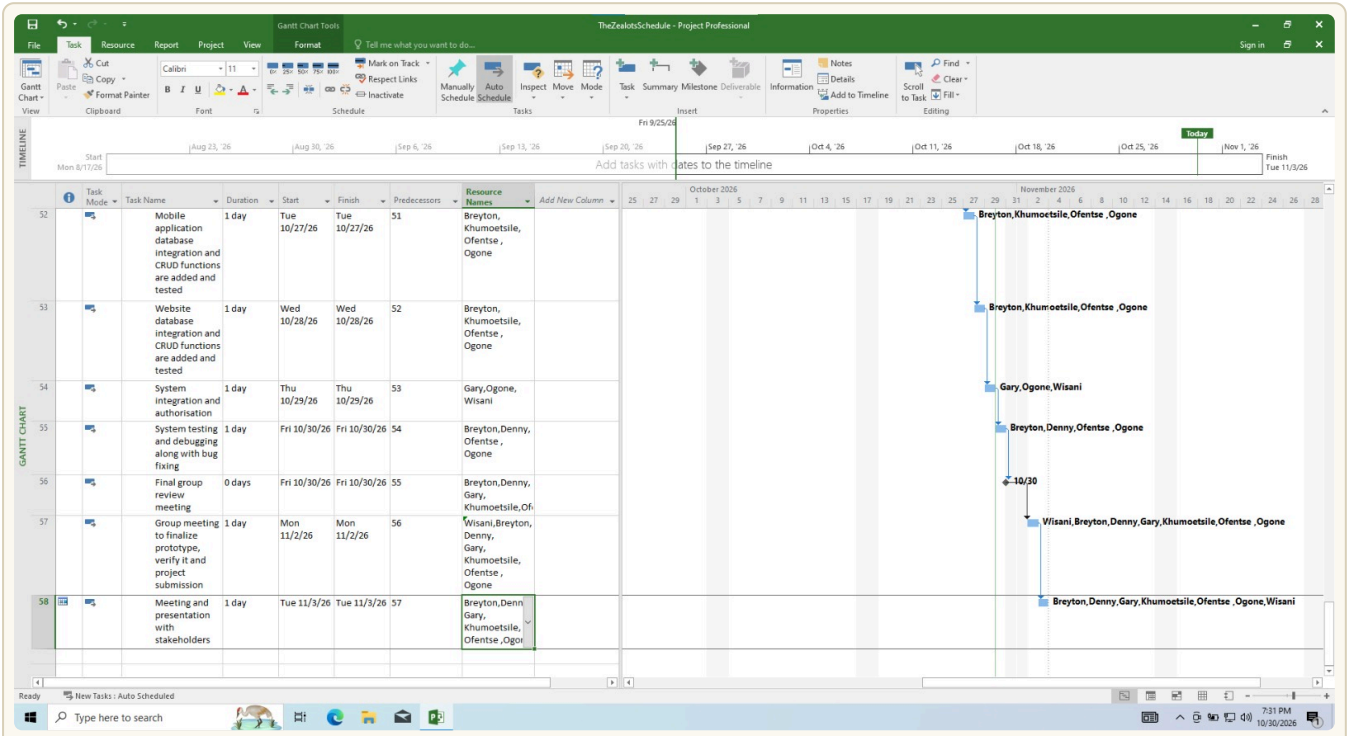
*This split follows the agreed demo plan; each member confirms their own line in the tracker.* Individual contribution rows are in the Phase 3 Progress Tracker; the commit history per member is on GitHub (Insights, Contributors).

# 8. Microsoft Project schedule

The editable schedule `TheZealotsSchedule (Phase 3, 30 Oct 2026).mpp` is in `project-docs/phase-3/schedule/` on GitHub. Phase 3 (task 45) runs 19 Oct to 3 Nov 2026: project kickoff, environment set-up, mobile and website development, backend and authentication, database integration with CRUD, system integration and authorisation, system testing and debugging (30 Oct), the final group review meeting (30 Oct), the finalisation meeting on 2 Nov and the stakeholder presentation on 3 Nov. The screenshots show the schedule with the Today marker on 30 Oct 2026.




Tasks 40 to 52: end of Phase 2 and the start of Phase 3.




Tasks 52 to 58: integration, testing, final review, submission and presentation.